



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет
«СТАНКИН»**

(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных
технологий

Кафедра
информационных систем

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

Отчет по дисциплине «Структуры и алгоритмы обработки данных»

Тема: «Лабораторная работа №3»

**Проверил
К.т.н., доцент**

Ковалев И.А.

подпись

**Выполнил
студент группы ИДБ-25-06**

Грачев А.А.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. 912. SORT AN ARRAY	4
2. 88. MERGE SORTED ARRAY	6
3. 56. MERGE INTERVALS	7
4. 347. TOP K FREQUENT ELEMENTS	8
5. 75. SORT COLORS	9
ЗАКЛЮЧЕНИЕ.....	10

ВВЕДЕНИЕ

Целью работы является закрепление навыков работы с массивами, сортировками и алгоритмами, использующими различные стратегии (жадные, сортировка по ключу, partition, гибридная сортировка). В отчёте рассмотрены пять задач с платформы LeetCode: 88. Merge Sorted Array, 912. Sort an Array, 56. Merge Intervals, 347. Top K Frequent Elements и 75. Sort Colors. В первой задаче была реализован помимо самого решения, была реализована функция, которая выбирает оптимальный алгоритм.

1. 912. SORT AN ARRAY

В данной задаче помимо решения самой задачи была реализована функция выбора оптимального алгоритма между insertion и merged sort.

Описание: Дан массив целых чисел, необходимо вернуть отсортированную копию.

Решение: Реализована гибридная стратегия. Для коротких массивов (≤ 32) или почти упорядоченных (менее 5% нарушений) используется сортировка вставками, в остальных случаях – сортировка слиянием. Такой выбор оптимизирует время для различных сценариев. Сложность в среднем $O(n \log n)$, в лучшем случае $O(n)$.

Код:

```
class Solution:
    def sortArray(self, nums: list[int]) -> list[int]:

        def insertion_sort(arr):
            for i in range(1, len(arr)):
                key = arr[i]
                j = i - 1
                while j >= 0 and arr[j] > key:
                    arr[j + 1] = arr[j]
                    j -= 1
                arr[j + 1] = key
            return arr

        def merge_sort(arr):
            if len(arr) <= 1:
                return arr
            mid = len(arr) // 2
            left = merge_sort(arr[:mid])
            right = merge_sort(arr[mid:])
            i = j = 0
            res = []
            while i < len(left) and j < len(right):
                if left[i] <= right[j]:
                    res.append(left[i])
                    i += 1
                else:
                    res.append(right[j])
                    j += 1
            res.extend(left[i:])
            res.extend(right[j:])
```

```
        return res

    def is_almost_sorted(arr):
        bad = 0
        for i in range(1, len(arr)):
            if arr[i] < arr[i - 1]:
                bad += 1
        return bad <= len(arr) * 0.05

    n = len(nums)
    if n <= 32:
        return insertion_sort(nums)
    if is_almost_sorted(nums):
        return insertion_sort(nums)
    return merge_sort(nums)
```

Тесты:

```
nums = [5, 2, 3, 1]
sol = Solution()
print(sol.sortArray(nums)) # [1, 2, 3, 5]
```

2. 88. MERGE SORTED ARRAY

Описание: Даны два отсортированных массива `nums1` (длины $m + n$, где последние n элементов — нули) и `nums2` (длины n). Требуется слить их в `nums1` в порядке неубывания, не возвращая новый массив.

Решение: Использован обратный проход с тремя указателями: $n1$ на последний значимый элемент `nums1`, $n2$ на последний элемент `nums2`, k на последнюю позицию результирующего массива. Сравниваем элементы и вставляем больший с конца. Сложность $O(m+n)$ по времени, $O(1)$ по памяти.

Код:

```
class Solution:
    def merge(self, nums1: list[int], m: int, nums2:
list[int], n: int) -> None:
        n1 = m - 1
        n2 = n - 1
        k = m + n - 1

        while n2 >= 0:
            if n1 >= 0 and nums1[n1] > nums2[n2]:
                nums1[k] = nums1[n1]
                n1 -= 1
            else:
                nums1[k] = nums2[n2]
                n2 -= 1
            k -= 1
```

Тесты:

```
nums1 = [1, 2, 3, 0, 0, 0]
m = 3
nums2 = [2, 5, 6]
n = 3
sol = Solution()
sol.merge(nums1, m, nums2, n)
print(nums1) # [1, 2, 2, 3, 5, 6]
```

3. 56. MERGE INTERVALS

Описание: Дан массив интервалов, представленных парами [start, end]. Необходимо объединить все пересекающиеся интервалы и вернуть непересекающийся массив.

Решение: Сортируем интервалы по началу. Затем идём по отсортированному списку и «склеиваем» текущий интервал с последним в результирующем списке, если они пересекаются. Это жадный алгоритм, работающий за $O(n \log n)$ из-за сортировки.

Код:

```
class Solution:
    def merge(self, intervals: list[list[int]]) ->
list[list[int]]:
        intervals.sort(key=lambda x: x[0])
        merged = []

        for interval in intervals:
            if not merged or merged[-1][1] < interval[0]:
                merged.append(interval)
            else:
                merged[-1][1] = max(merged[-1][1],
interval[1])

        return merged
```

Тесты:

```
intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]
sol = Solution()
print(sol.merge(intervals)) # [[1, 6], [8, 10], [15, 18]]
```

4. 347. TOP K FREQUENT ELEMENTS

Описание: Дан массив `nums` и число `k`. Необходимо вернуть `k` наиболее часто встречающихся элементов в любом порядке.

Решение: Подсчитываем частоты элементов с помощью `Counter`. Затем сортируем пары (элемент, частота) по убыванию частоты (сортировка по ключу) и берём первые `k` элементов. Время $O(n \log n)$ из-за полной сортировки, память $O(n)$.

Код:

```
from collections import Counter

class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        freq = Counter(nums)
        sorted_items = sorted(freq.items(), key=lambda x: x[1], reverse=True)
        return [item[0] for item in sorted_items[:k]]
```

Тесты:

```
nums = [1, 1, 1, 2, 2, 3]
k = 2
sol = Solution()
print(sol.topKFrequent(nums, k)) # [1, 2]
```


5. 75. SORT COLORS

Описание: Массив `nums` содержит только 0, 1 и 2. Необходимо отсортировать его `in-place` так, чтобы одинаковые элементы шли подряд (порядок: 0, 1, 2).

Решение: Задача решается за один проход с помощью метода трёх указателей (Dutch National Flag problem). Указатель `low` отделяет нули, `high` – двойки, `mid` сканирует массив. Если `nums[mid] == 0`, меняем с `low` и сдвигаем оба; если 1, просто идём дальше; если 2, меняем с `high`. Время $O(n)$, память $O(1)$.

Код:

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        low, mid, high = 0, 0, len(nums) - 1

        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low]
                low += 1
                mid += 1
            elif nums[mid] == 1:
                mid += 1
            else: # nums[mid] == 2
                nums[mid], nums[high] = nums[high], nums[mid]
                high -= 1
```

Тесты:

```
nums = [2, 0, 2, 1, 1, 0]
sol = Solution()
sol.sortColors(nums)
print(nums) # [0, 0, 1, 1, 2, 2]
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были реализованы и протестированы пять алгоритмов, охватывающих основные техники работы с массивами и сортировками: слияние отсортированных массивов с конца, гибридная сортировка с выбором стратегии, жадное объединение интервалов после сортировки, сортировка по ключу для выбора наиболее частых элементов и in-place partition (метод трёх указателей) для сортировки трёх цветов. Каждый алгоритм проанализирован по временной и пространственной сложности, а также проверен на тестовых примерах, предоставленных платформой LeetCode.